TDSTR Reference Manual

Tcl Front End and Tooling

Sambuddha Majumder and Jayanta Majumder

May 25, 2026

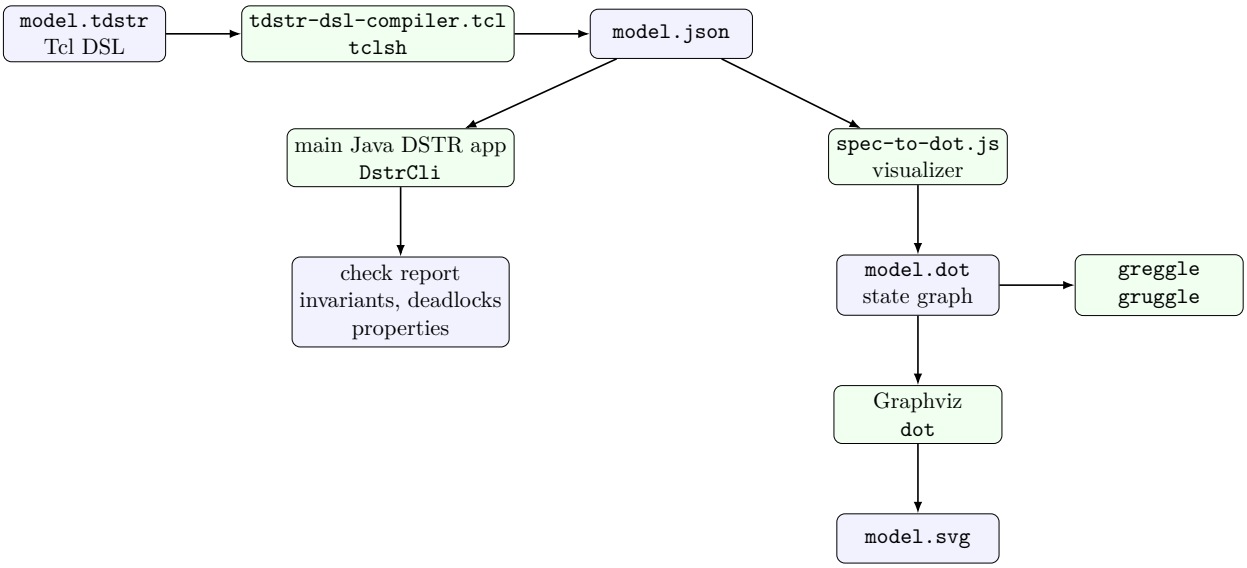
Contents

1 Purpose and Scope

TDSTR is the Tcl-shaped front end for the `dstr` finite-state modelling prototype. It uses Tcl's command and brace syntax to describe the same model concepts as DSTR and CDSTR: finite variables, domains, initial states, actions, the next-state relation, invariants, and reachability-style properties. The TDSTR compiler emits normalized JSON for the common checker and graph tools.

TDSTR is a good fit when a Tcl interpreter is available and the model benefits from Tcl's lightweight command language, `proc` definitions, and `sourced` helper files.

2 Pipeline



The Tcl compiler is a front end. Its JSON output should normally be passed to the root Java DSTR application, `org.dstr.cli.DstrCli`, for actual model checking. The DOT/SVG path is the visualization path: useful for inspecting small graphs, debugging transitions, and preparing

diagrams. The generated `.dot` graph can also be used as input to `greggle` and `gruggle` for graph interrogation and manipulation.

3 Command Reference

3.1 Compile a File

Run the compiler directly with Tcl:

```
tclsh scripts/tdstr-dsl-compiler.tcl \
  tdstr-testsuite/light-switch.tdstr \
  target/light-switch.json
```

PowerShell:

```
tclsh scripts\tdstr-dsl-compiler.tcl ‘
  tdstr-testsuite\light-switch.tdstr ‘
  target\light-switch.json
```

3.2 Compile a Directory

```
tclsh scripts/tdstr-dsl-compiler.tcl tdstr-testsuite target/tdstr-json
```

Every `.tdstr` file in the input directory is compiled to a corresponding `.json` file in the output directory.

3.3 Run the Java Model Checker

After compiling to JSON, run the main Java checker from the repository root:

```
mvn exec:java -Dexec.args="target/light-switch.json"
gradle run --args="target/light-switch.json"
```

The checker parses the JSON emitted by TDSTR, enumerates the finite state space, computes reachable states from `init` through `next`, checks invariants, reports deadlock counterexample paths, and evaluates `eventually` properties as existential reachability summaries. Use this path for the authoritative pass/fail checking result.

3.4 Generate SVG

```
./tdstr-to-svg.sh tdstr-testsuite/light-switch.tdstr
./tdstr-to-svg.sh tdstr-testsuite/bakery-3proc.tdstr --max-states 500
```

```
.\tdstr-to-svg.bat tdstr-testsuite\light-switch.tdstr
.\tdstr-to-svg.bat tdstr-testsuite\bakery-3proc.tdstr --max-states 500
```

The wrapper accepts `input.tdstr` or `input.json`. It writes adjacent `.json`, `.dot`, and `.svg` files. It is meant for graph inspection and does not replace the Java model-checking CLI. The `.svg` is the visual rendering; the `.dot` graph is also suitable as input to `greggle` or `gruggle`.

3.5 Graph Options

`-all-states` Include unreachable states.

`-rankdir TB` Draw top-to-bottom. Use `LR` for left-to-right.

`-max-states N` Truncate very large universes.

`-compact` Force compact graph labels.

`-no-legend` Omit compact graph legend.

4 Tcl Source Model

A TDSTR file is a Tcl script evaluated in an isolated namespace. The compiler installs DSL commands such as `system`, `vars`, `domain`, and `action` into that namespace. Ordinary Tcl `proc` definitions may appear before a `system`, and helper files may be loaded with `source`. Relative `source` paths are resolved against the current `.tdstr` file's directory. Java and protocol buffer enum files may also be loaded before the `system` with `load-java-enums` and `load-proto-enums`; relative paths are resolved from the current TDSTR file.

Use braces to prevent early Tcl substitution in model expressions:

```
action turn-on {= light off} {= light+ on}
```

Use command substitution deliberately when invoking helper procedures:

```
action p1-a0 {= p1pc a0} {= p1pc+ a1} {*}[unchanged p2pc c1 c2 turn]
```

5 System Form

```
system light-switch {
    vars light
    domain light off on
    init {= light off}
    action turn-on {= light off} {= light+ on}
    action turn-off {= light on} {= light+ off}
    next {or @turn-on @turn-off}
    invariant type-ok {in light {set off on}}
    property eventually-on {eventually {= light on}}
}
```

5.1 Clauses

`vars v...` Declare variables.

`vars* sep {a...} * {b...}` Declare product variables by joining each pair with `sep`.

`domain v value...` Declare a finite domain for one variable. A single `@EnumName` value expands to the symbols loaded from a Java enum type.

`domain* pattern value...` Apply a domain to all declared variables matching a Tcl glob pattern.

`init expr...` Define initial-state predicate.

`action name expr...` Define a named transition predicate.

`next expr...` Define next-state relation, usually action refs.

`invariant name expr...` Check a predicate on reachable states.

`property name expr...` Summarize a property such as `eventually`.

Multiple body expressions in a clause are combined as `and`.

6 Atoms and References

Current variable A declared name such as `pc`.

Next variable A declared name with trailing `+`, such as `pc+`.

Action reference A token beginning with `@`, such as `@turn-on`, in a `next` expression.

String literal An undeclared word such as `idle`, `OPEN`, or `CANCELLED`.

Numeric literal Integers and floating point tokens.

Boolean literal `true`, `false`, `t`, and `nil`.

Quoted literal `{quote foo}` forces a literal.

7 Expression Reference

7.1 Logic

```
{and {= pc idle} {= lock free}}
{or @enter @exit}
{not {= pc cs}}
{implies {= pc cs} {= lock held}}
```

7.2 Comparison and Arithmetic

```
{= count 0}
{!= mem 0}
{< ticket other-ticket}
{<= hr 12}
{= count+ {+ count 1}}
{= remaining+ {- remaining 1}}
```

7.3 Sets and Membership

```
domain pc idle trying cs
invariant type-ok {in pc {set idle trying cs}}
```

7.4 Domains from Java Enums

`load-java-enums` accepts one or more Java files or directories before the `system` command. Directories are searched recursively. The parser targets ordinary Java enum declarations with comma-separated constants, allowing whitespace and comments between constants.

```
load-java-enums src/main/java

system lifecycle-model {
    vars state
    domain state @Lifecycle
    init {= state new}
    action activate {= state new} {set state as ready}
    next @activate
}
```

TDSTR normalizes enum constants to lower-case strings, matching the rest of the TDSTR compiler's symbol handling.

7.5 Domains from Protocol Buffer Enums

`load-proto-enums` accepts one or more `.proto` files or directories before the `system` command. Directories are searched recursively. The parser recognizes entries of the form `NAME = number;` and ignores comments, enum options, reserved declarations, and option values.

```
load-proto-enums src/main/proto

system order-model {
    vars status
    domain status @OrderStatus
    init {= status order_status_new}
    action ready {= status order_status_new} {set status as order_status_ready}
    next @ready
}
```

TDSTR normalizes protobuf enum symbols to lower-case strings.

7.6 Quantifiers

```
invariant known-owner {
    forall {p in {set p1 p2}}
        {in p {set p1 p2}}
}

property owner-can-be-p1 {
    eventually {exists {p in {set p1 p2}} {= p p1}}
}
```

7.7 Reachability-Style Temporal Form

```
property p1-can-reach-cs {eventually {= p1pc cs}}
```

The current checker treats this as reachability, not fairness-based temporal logic.

8 Transition Sugar

8.1 Assignments

```
{= light+ on}
{assign on to light}
{set light as on}
```

All three assert equality between the next state of `light` and `on`.

8.2 Unconditional Transitions

An action body is a conjunction of its expression arguments. For an unconditional transition, write the next-state assignments directly instead of using `if ... then ...`:

```
action reset \
    {assign IDLE to pc} \
    {set count as 0} \
    {assign false to busy}
```

The same transition can be written with explicit next-state equalities:

```
action reset \
    {= pc+ IDLE} \
    {= count+ 0} \
    {= busy+ false}
```

If some variables should stay unchanged, include frame conditions too:

```
action reset \
    {assign IDLE to pc} \
    {set count as 0} \
    {*}[unchanged owner mode]
```

or with glob-based frame sugar:

```
action reset \
    {assign IDLE to pc} \
    {set count as 0} \
    {unchanged* *_status *_version}
```

If an action leaves a next-state variable unconstrained, the checker may consider every value in that variable's domain as a possible next value.

8.3 Equality Alias

```
{equals p_settlement_state OPEN}
```

is equivalent to:

```
{= p_settlement_state OPEN}
```

8.4 Guarded Transition Cases

```
{
    if
    {
        {equals p_settlement_state OPEN}
        {equals c1_settlement_state OPEN}
        {equals c2_settlement_state OPEN}
    }
    then
    {
        {assign CANCELLED to p_settlement_state}
        {assign CANCELLED to c1_settlement_state}
        {assign CANCELLED to c2_settlement_state}
    }
}
```

The `if ... then ...` form is a guarded-conjunction shorthand. It is equivalent to `and` of the condition block and the consequence block. There is no `else` clause.

8.5 Frame Conditions

```
{unchanged* *_market_status *_event_type *_mission_version *_mission_revision}
```

The compiler expands all matching declared variables into frame equalities.

8.6 Alternate Scenarios

```
{
    alternate-scenarios
    {if {= state OPEN} then {assign CANCELLED to state}}
    {if {= state BOOKED} then {assign state to state}}
}
```

`alternate-scenarios` is a synonym for `or`.

9 Built-In Tcl Helpers

9.1 `same`

```
same pc
```

returns:

```
{= pc+ pc}
```

9.2 `unchanged`

```
action p1-a0 {= p1pc a0} {= p1pc+ a1} {*}[unchanged p2pc c1 c2 turn]
```

`unchanged` returns a Tcl list of sibling frame-condition expressions. The Tcl expansion operator `{*}` splices them into the action command.

9.3 Custom Helpers

```
proc preservePair {left right} {
    return [list [same $left] [same $right]]
}

system example {
    vars left right
    domain left idle done
    domain right idle done
    init {= left idle} {= right idle}
    action finish-left {= left idle} {= left+ done} {*}[preservePair right right]
    next @finish-left
}
```

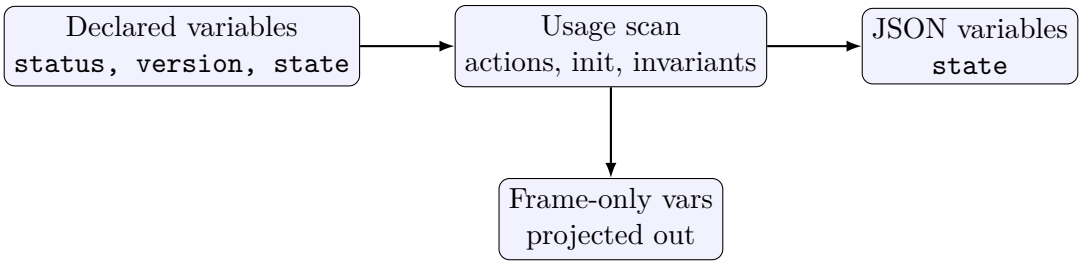
Since TDSTR is Tcl, helpers can compute and return expression lists. Keep helper output in the same brace/list shape that the compiler expects.

10 Product Variables and Domain Globs

```
system lifecycle {
    vars* _ {master venue1} * {lifecycle trading_status}
    domain* *_lifecycle NEW ENRICHING READY DISABLED
    domain* *_trading_status NOT_TRADING TRADING PAUSED

    init \
        {equals master_lifecycle ENRICHING} \
        {equals venue1_lifecycle ENRICHING}

    action enrichment-complete \
        {if
            {
                {equals master_lifecycle ENRICHING}
                {equals venue1_lifecycle ENRICHING}
            }
            then
            {
                {assign READY to master_lifecycle}
                {assign READY to venue1_lifecycle}
                {assign NOT_TRADING to master_trading_status}
                {assign NOT_TRADING to venue1_trading_status}
            }}

    next @enrichment-complete
}
```

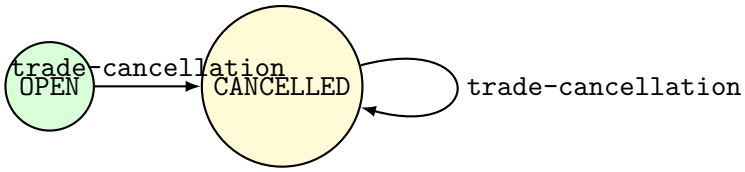
11 Frame-Only Variable Reduction

The TDSTR compiler performs a safe post-expansion reduction. Declared variables that occur only in conjunctive unchanged/frame clauses are omitted from the emitted JSON. This reduces state-space size for models with many fields that are carried along unchanged.



The graph generator reports projected-out variables in graph labels when that information is visible through the normalized spec.

12 State Graph Output



Green nodes are initial states, amber nodes are deadlocks, and red nodes indicate invariant failures. Edges are labelled by actions that justify each transition.

13 Full Example: Small Trade Cancellation

```
system trade-cancellation-parent-children-small {
    vars* _ {p c1 c2} * {market_status settlement_state settlement_version
    settlement_revision event_type}

    domain* *_market_status MATCHED
    domain* *_settlement_state OPEN RELEASED BOOKED CANCELLED
    domain* *_event_type INSTRUCTION_ACKED
    domain* *_mission_version 1
    domain* *_mission_revision 1

    init \
        {equals p_settlement_state OPEN} \
        {equals c1_settlement_state OPEN} \
        {equals c2_settlement_state OPEN}

    action trade-cancellation \
        {unchanged* *_market_status *_event_type *_mission_version *_mission_revision} \
        {
            alternate-scenarios
            {
                if
                {
                    {equals p_settlement_state OPEN}
                    {equals c1_settlement_state OPEN}
                    {equals c2_settlement_state OPEN}
                }
```

```
                then
                {
                    {assign CANCELLED to p_settlement_state}
                    {assign CANCELLED to c1_settlement_state}
                    {assign CANCELLED to c2_settlement_state}
                }
            }
        }

    next @trade-cancellation
}
```

14 Practical Modelling Notes

- Use braces for DSL expressions so Tcl does not substitute variables or commands too early.
- Use `@action` for TDSTR action references. The lispy `$action` convention is not used in TDSTR.
- Use `{*}[helper ...]` when a helper returns multiple sibling expressions.
- Be explicit about frame conditions. Unconstrained next-state variables create extra successor states.
- Use `domain*` and `unchanged*` together for large product models.
- Use `-max-states` while visualizing large state spaces.

15 Checklist

1. Write a `system name {...}` block.
2. Declare variables with `vars` or `vars*`.
3. Provide domains with `domain` or `domain*`.
4. Define `init`.
5. Define transition `action` clauses.
6. Define `next`, normally with `@action` references.
7. Add invariants and `eventually` properties where useful.
8. Compile with `tclsh scripts/tdstr-dsl-compiler.tcl`.
9. Run the Java DSTR checker on the generated JSON for the checking report.
10. Render with `tdstr-to-svg` or `spec-to-dot.js` when a graph view is useful.